PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)

ON

NIX - AI Coding Assistant

submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Nikhil Saini (2211981245)

Akshit Mehta (2211981043)

Supervised By:

Dr. Rajat Takkar

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

DECLARATION

I hereby certify that the work which is being presented in the project report entitled **" NIX - AI Coding Assistant "** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Nikhil Saini (2211981245)

Akshit Mehta (2211981043)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We are grateful to our guide Dr. Rajat Takkar for his continuous support, guidance, and valuable feedback throughout the course of this project. His encouragement has been instrumental in helping us complete this work successfully.We also extend our gratitude to Chitkara University for providing us the opportunity and resources to work on this project. Finally, we thank our parents and friends for their constant motivation and support.

CONTENTS

Abstract

Spring Boot is widely used for developing scale-able and efficient back-end applications due to its modular architecture and ease of integration. However, as projects grow in size and complexity, developers face significant challenges in understanding code structure, managing dependencies, and analyzing service interactions. Large-scale applications often consist of multiple interconnected components such as controllers, services, repositories, and configuration files, making it difficult to trace the flow of execution and identify relationships between different modules. This complexity increases the time required for debugging, feature enhancement, and on-boarding new developers.

Traditional development tools such as Integrated Development Environments (IDEs) provide features like syntax highlighting, code navigation, and debugging support. However, they often lack the capability to provide a holistic and contextual understanding of the entire system. Developers are required to manually explore files, search for references, and interpret logic, which leads to reduced productivity and increased cognitive load.

Nix is designed to overcome these limitations by introducing an AI-powered assistant that operates directly within the terminal environment. It enables developers to interact with their code-base using natural language queries and receive structured, context-aware responses based on actual code analysis rather than assumptions. By leveraging technologies such as Abstract Syntax Tree (AST) parsing and semantic search, Nix ensures accurate and meaningful insights.

The system focuses on improving developer productivity, reducing debugging time, and simplifying codebase exploration while ensuring complete data privacy through local execution. Additionally, it supports interactive multi-turn queries, minimizes cognitive effort, and enhances overall development efficiency by enabling faster decision-making and streamlined workflows.

Keywords: AI Coding Assistant, Spring Boot, AST Parsing, Semantic Search, ChromaDB, Groq LLM, CLI Tool, Code Analysis, Developer Productivity, Privacy-Preserving AI.

1. Introduction

Software development, especially backend development using frameworks like Spring Boot, has become increasingly complex due to the growing scale and modular design of modern applications. Developers are required to manage multiple interconnected components such as REST APIs, service layers, databases, and configuration files. As projects expand, understanding the relationships between these components and tracing the flow of execution becomes a challenging and time-consuming task. This complexity often leads to increased debugging effort, slower development cycles, and difficulties in onboarding new developers.

Traditional tools like Integrated Development Environments (IDEs) provide basic support for navigation and debugging but lack the capability to offer a comprehensive and contextual understanding of the entire codebase. Developers must manually search through files, follow dependencies, and interpret logic, which increases cognitive load and reduces overall productivity. Additionally, many AI-based solutions rely on cloud processing, requiring developers to share source code externally, which raises serious concerns regarding data privacy and security.

To address these challenges, Nix is introduced as a local, AI-powered command-line assistant designed specifically for Spring Boot developers. It enables users to interact with their codebase using natural language queries and receive structured, accurate insights based on actual code analysis. By combining static code parsing with intelligent query processing, Nix simplifies code exploration, enhances debugging efficiency, and ensures complete data privacy by operating entirely within the local environment.

The system focuses on improving developer productivity, reducing debugging time, and simplifying codebase exploration while ensuring complete data privacy through local execution. Furthermore, it supports interactive multi-turn conversations, minimizes cognitive effort, and enhances overall development efficiency by enabling faster decision-making, streamlined workflows, and a more intuitive approach to understanding complex software systems.

1.1 Problem Background

In large-scale Spring Boot applications, developers often face difficulties in:

1. Understanding bean dependencies and service relationships
2. Identifying REST endpoint mappings across multiple controllers
3. Tracing database entity relationships
4. Debugging issues caused by complex inter-service interactions

Existing tools provide limited automation in these areas and rely heavily on manual exploration. Additionally, AI-based tools available today often require uploading code to cloud services, which introduces risks related to data privacy and intellectual property.

This creates a need for a system that is both intelligent and secure, capable of providing deep insights into the code-base without external dependencies.

1.2 Objective

The primary objectives of this project are:

1. To design and develop a local AI-powered assistant for code analysis
2. To enable natural language interaction with Spring Boot applications
3. To reduce manual effort in debugging and code exploration
4. To ensure data privacy by avoiding external data transmission
5. To provide structured and accurate insights based on actual code
6. To enhance developer productivity and reduce onboard time.

2. Methodology

The methodology of the system combines static code analysis with artificial intelligence to improve the developer experience. The process starts by parsing the Spring Boot project using Tree-sitter, which converts Java code into an Abstract Syntax Tree (AST). This helps in identifying important components such as controllers, services, repositories, REST endpoints, and entities. The extracted data is then stored in a local JSON index for quick and efficient access.

To make searching smarter, ChromaDB is used to create vector embeddings, allowing the system to understand the meaning of queries even if exact keywords are not used. When a user asks a question, a conversational agent processes the query, selects the appropriate tool, and retrieves relevant information from the stored data.
The Groq LLM then generates a clear and structured response based on the actual codebase. The system also supports follow-up queries, making it easier for users to explore the project step by step.

Step 1: Code Parsing:The system begins by analyzing the Spring Boot project using Tree-sitter. This tool parses Java source code and generates an Abstract Syntax Tree (AST), which represents the structure of the code in a hierarchical format.

Step 2: Data Extraction: From the AST, relevant components such as:

- Controllers
- Services
- Repositories
- Beans
- REST endpoints
- JPA entities

Step 3: Indexing: The extracted information is stored in a local JSON-based index. This ensures fast retrieval of code-related data without re-parsing the codebase repeatedly.

Step 4: Semantic Search : Chroma Db is used to generate vector embeddings for the code elements. This enables semantic search, allowing the system to understand queries even when exact keywords are not used.

Step 5: Query Processing: When a user enters a query, the system:

- Interprets the query using a conversational agent
- Identifies the appropriate analysis tool
- Retrieves relevant data from the index or vector database

Step 6: Response Generation

The Groq LLM processes the retrieved data and generates a context-aware response. The response is structured and aligned with the actual codebase.

Step 7: Multi-turn Interaction

The system supports follow-up queries, allowing users to refine their questions and explore the codebase interactively.

3. Tools and Technologies

The project uses a combination of modern tools and technologies:

- Python: Used for implementing core logic due to its flexibility and strong ecosystem
- Tree-sitter: Provides efficient parsing of Java code into AST format
- ChromaDB: Enables vector-based semantic search across the codebase
- Groq LLM API: Delivers fast and context-aware AI-generated responses
- CLI Interface: Ensures lightweight and efficient user interaction
- JSON Storage: Used for maintaining indexed code structure
- AST (Abstract Syntax Tree): Forms the backbone of code analysis

These technologies work together to provide accurate, fast, and scalable performance.

4. Implementation

The implementation of Nix follows a modular architecture:

1. Parser Module: Responsible for analyzing Java code using Tree-sitter and generating AST structures.

2. Indexing Module: Stores extracted data in JSON format, ensuring efficient lookup and retrieval.

3. Vector Database Module**:** Uses ChromaDB to store embeddings and enable semantic similarity searches.

4. AI Agent Module: Acts as the core component that processes user queries, selects tools, and generates responses.

5. Tool Engine: Contains 24 specialized tools designed for:

1. Dependency analysis
2. Endpoint discovery
3. Entity mapping
4. Call chain tracing

6. CLI Interface: Provides a user-friendly command-line interface where developers can interact with the system.

The modular design ensures scalability and allows easy integration of additional features in the future.

5. Major Findings & Output

The implementation of Nix resulted in several significant improvements in the software development workflow, particularly for Spring Boot applications. The system effectively reduces the time required to understand complex codebases by providing direct, structured insights into various components such as dependencies, endpoints, and service interactions. Developers no longer need to manually navigate through multiple files, which significantly improves debugging efficiency and reduces development time.

The tool enhances developer productivity by minimizing repetitive tasks such as searching for references, tracing method calls, and identifying relationships between components. By enabling natural language interaction, Nix allows developers to quickly retrieve relevant information, making the development process more intuitive and efficient. It also supports faster onboarding, as new developers can understand unfamiliar codebases more easily without requiring extensive guidance or documentation.

Another key outcome is the accuracy of responses generated by the system. Since the outputs are based on actual code structure using AST parsing, the information provided is reliable and context-aware rather than generic. This ensures that developers can make informed decisions during debugging and development.

In addition, the local-first architecture of Nix ensures secure processing, as no source code is transmitted to external servers. This makes the system suitable for enterprise environments where data privacy is critical. The system also demonstrates fast response times due to local execution and efficient indexing mechanisms.

As observed in the result , the system successfully processes queries such as dependency analysis and provides structured outputs directly in the terminal. Overall, Nix improves efficiency, accuracy, and usability in backend development, making it a valuable tool for modern software engineering workflows.

5.1 Output

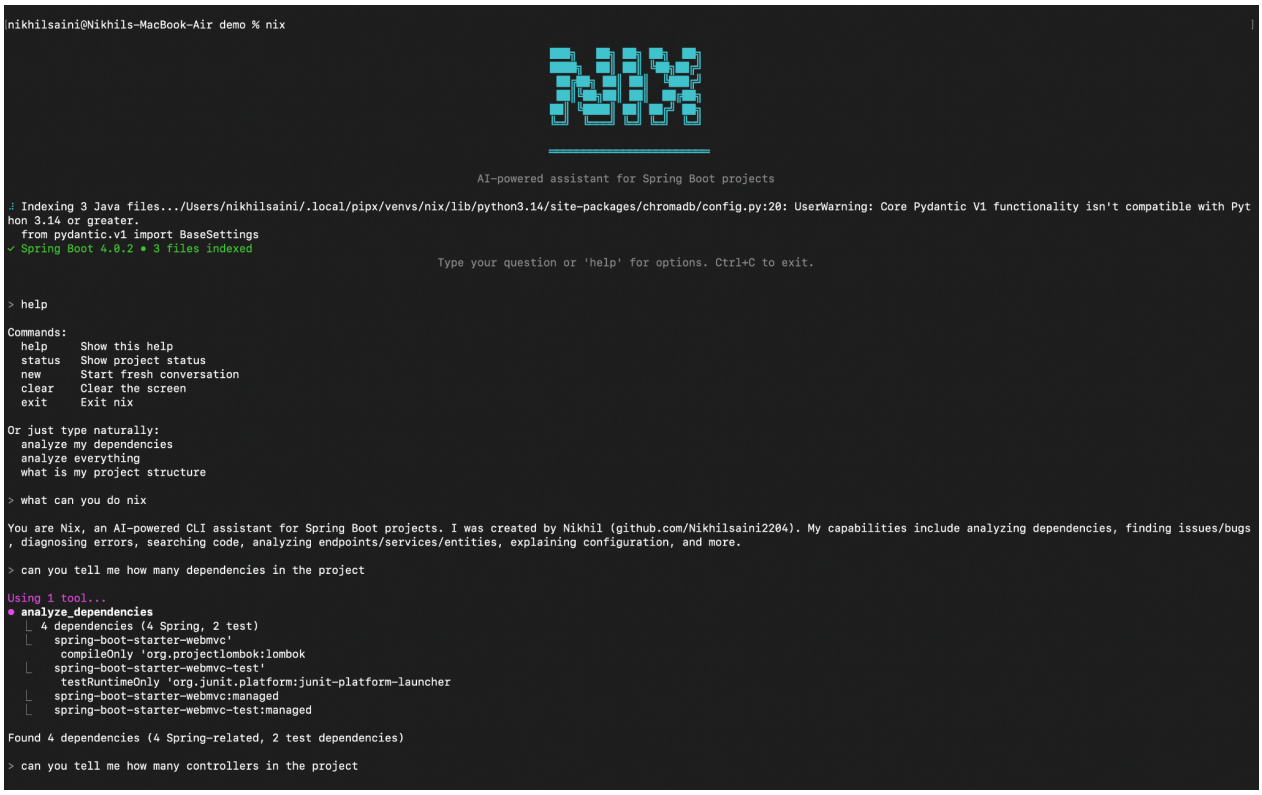

Figure 5.1

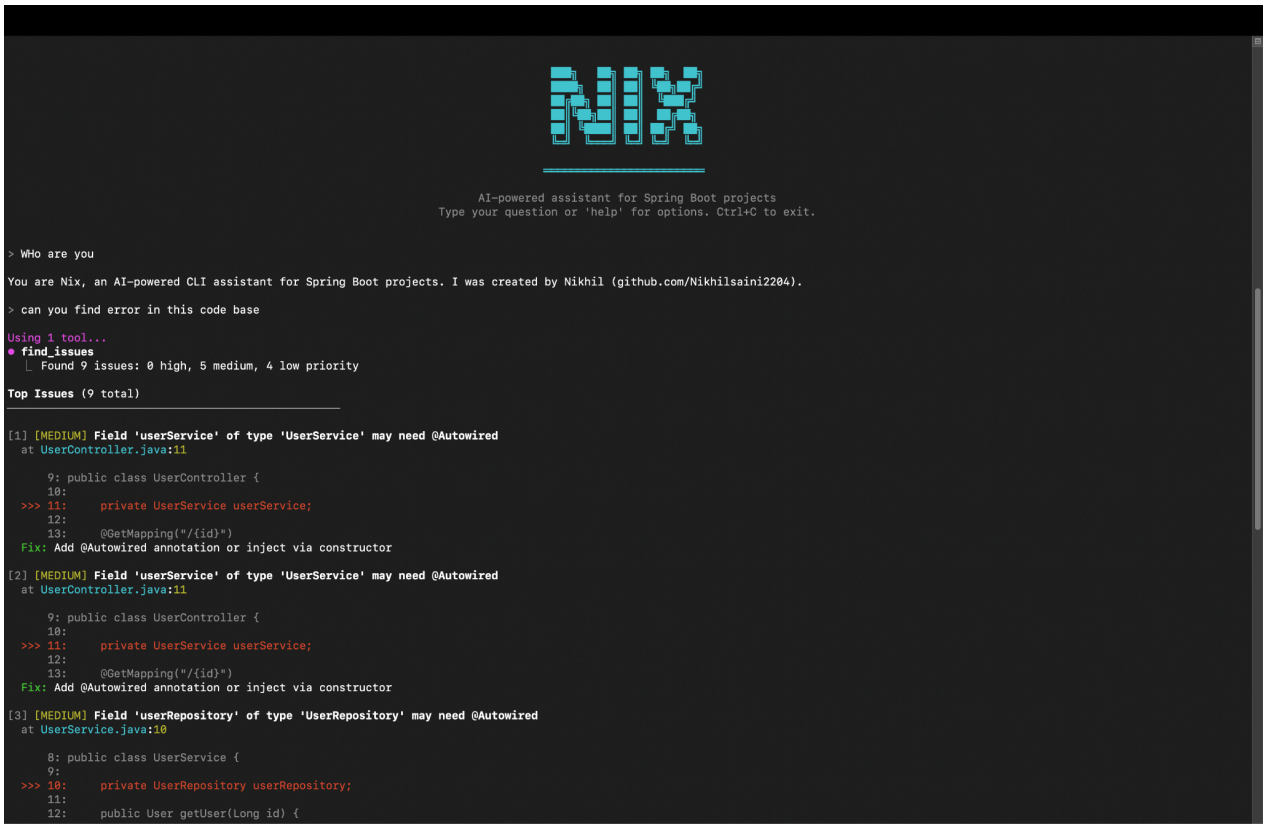
AI-powered assistant for Spring Boot projects
Type your question or 'help' for options. Ctrl+C to exit.
> WHo are you
You are Nix, an AI-powered CLI assistant for Spring Boot projects. I was created by Nikhil (github.com/Nikhilsaini2204).
> can you find error in this code base
Using 1 tool...
● find_issues
Found 9 issues: 0 high, 5 medium, 4 low priority
Top Issues (9 total)
[1] [MEDIUM] Field 'userService' of type 'UserService' may need @Autowired
at UserController.java:11
9: public class UserController {
10:
>>> 11: private UserService userService;
12:
13: @GetMapping("/{id}")
Fix: Add @Autowired annotation or inject via constructor
[2] [MEDIUM] Field 'userService' of type 'UserService' may need @Autowired
at UserController.java:11
9: public class UserController {
10:
>>> 11: private UserService userService;
12:
13: @GetMapping("/{id}")
Fix: Add @Autowired annotation or inject via constructor
[3] [MEDIUM] Field 'userRepository' of type 'UserRepository' may need @Autowired
at UserService.java:10
8: public class UserService {
9:
>>> 10: private UserRepository userRepository;
11:
12: public User getUser(Long id) {

Figure 5.2

6. Conclusion

The Nix project demonstrates the effective integration of artificial intelligence with static code analysis to address real-world challenges in software development. By enabling natural language interaction with codebases, it simplifies complex tasks such as debugging, dependency tracing, and system understanding. The system provides developers with structured and context-aware insights, reducing the need for manual code exploration and improving overall development efficiency.

One of the key strengths of Nix lies in its ability to bridge the gap between traditional development tools and modern AI-driven solutions. Unlike conventional tools that require extensive manual effort, Nix offers an intuitive and interactive approach to code analysis. Its ability to process queries in natural language makes it accessible to both experienced developers and beginners, thereby enhancing usability and reducing the learning curve.

The local-first design ensures complete data privacy, as all processing is performed within the developer's environment. This makes the system highly suitable for enterprise-level applications where security and confidentiality are critical. Additionally, the use of technologies such as AST parsing and semantic search ensures that the responses generated are accurate, relevant, and aligned with the actual codebase.

Furthermore, the project highlights the potential of combining AI with software engineering practices to build intelligent developer tools. It not only improves productivity but also contributes to faster debugging, better code understanding, and efficient onboarding processes.

Overall, Nix enhances developer efficiency and provides a modern, scalable, and secure approach to backend development. It serves as a strong foundation for future advancements in AI-assisted programming and demonstrates how intelligent systems can transform traditional development workflows into more streamlined and effective processes.

7. Future Scope

The project can be further enhanced by expanding its capabilities and improving usability to make it more versatile and industry-ready. One important enhancement is supporting additional backend frameworks such as Quarkus, Micronaut, and Node.js-based architectures, allowing the system to be used across a wider range of development environments. Developing a graphical user interface (GUI) can further improve usability by providing visual representations of code structures, making the tool more accessible to users who prefer graphical interaction over command-line interfaces.

Another major improvement is the integration of fully offline large language models using tools like Ollama or LM Studio. This would eliminate dependency on external APIs and further strengthen data privacy and security. The system can also be extended to include advanced code quality analysis and performance optimization features, helping developers identify code smells, inefficiencies, and potential improvements.

Visualization of system architecture, including dependency graphs and service interaction maps, can provide a clearer understanding of complex codebases. Support for microservices-based architectures can enable analysis across multiple services, including API communication and distributed dependencies.

Additionally, automated documentation generation can be implemented to create technical documentation directly from the codebase, reducing manual effort. The system can also include intelligent test case generation and debugging suggestions based on code analysis. Collaborative features, such as shared indexed codebases and team-based query systems, can enhance teamwork and knowledge sharing within development teams.

Overall, these enhancements can transform Nix into a comprehensive AI-driven development assistant, making it more powerful, scalable, and suitable for real-world enterprise applications

8. References

1. Spring Boot Official Documentation – https://spring.io/projects/spring-boot
2. Tree-sitter Documentation – https://tree-sitter.github.io/tree-sitter/
3. ChromaDB Official Documentation – https://docs.trychroma.com/
4. Groq LLM API Documentation – https://console.groq.com/docs
5. GitHub Repository – Nix Project – https://github.com/Nikhilsaini2204/nix
6. Research Papers on AI in Software Engineering (IEEE, Springer, ACM Digital Library)
7. Java Programming Documentation – Oracle Official Documentation
8. Natural Language Processing (NLP) Concepts – Stanford NLP Resources
9. Vector Databases and Embedding Techniques – OpenAI / Research Blogs
10. Code Analysis and Static Analysis Techniques – Research Journals and Technical Articles
11. Documentation on Command-Line Interface (CLI) Design and Development

9. Appendices

1. Sample natural language queries and corresponding system responses
2. Code snippets of core modules including parser, indexing, and query engine
3. GitHub repository link for full project source code
4. System architecture diagrams illustrating workflow and data flow
5. Flowcharts representing system design and processing pipeline
6. Sample JSON index structure generated from code parsing
7. Example outputs of dependency analysis and endpoint discovery
8. Performance observations and response time analysis
9. User interaction examples showcasing multi-turn conversations
10. Error handling cases and system response behavior
11. Installation and setup instructions for running the project locally